Higher Technological Institute
FACULITY OF ENGINEERING
ELECTRICALCOMMUNICATIONS DEPARTMENT

Smart Traffic Light Control System with Temperature and Humidity Monitoring using ATmega32

ComputerMicroprocessor
EEC214

Submitted By:

Asmaa Khalid Moustafa	**20220693**	**Group 3**
Areeg Moustafa Mohamed	**20223115**	**Group 3**
Mohamed Ahmed Salah	**20243150**	**Group 3**
Alaa Mohamed Abdo	**20243301**	**Group 3**

Submitted To:

Prof.Dr. Mohamed Torad

2026

Abstract

This project presents the design and implementation of a smart traffic light control system for a four-road intersection using the ATmega32. The system controls traffic flow by operating four traffic signals consisting of red, yellow, and green LEDs. Each traffic signal includes a countdown timer displayed on two 7-segment displays to inform drivers of the remaining time before the signal changes.

The traffic light operates in a timed sequence where the green signal remains active for 37 seconds followed by a yellow warning period of 3 seconds, while the red signal remains active for the opposing roads. Additionally, the system includes an environmental monitoring feature using DHT11 to measure temperature and humidity. The measured values are continuously displayed on an LCD 16x2 using an I2C communication interface.

Furthermore, the LCD displays a set of safety warning messages that change every five seconds to improve driver awareness and road safety. The system is designed and tested using Proteus before hardware implementation. This project demonstrates how embedded systems can be used to improve traffic management and provide additional environmental information to drivers.

Acknowledgement

At first, thanks to ALLAH, the most merciful, the most gracious, for this moment has come and this work has been accomplished. Thanks to the Higher Technological Institute of 10th Ramadan for preparing me to be a successful engineer and lifting me up to achieve this training in an environment that's full of encouragement and motivation.

My deepest thanks to my parents, whose sacrifices, endless support, and belief in my abilities have been the foundation of my educational journey. Their prayers and continuous encouragement have always been my source of strength and motivation.

My highest gratitude to the faculty members of the Department of Electrical & Computers Engineering for all the support they provide us during the period of study at the Institute. Special thanks to instructor **Prof.Dr. Mohammed Torad** for her help and knowledge in the field of microcontroller. His professional touches are sensed within every phase of this Computer Microprocessor course.

Table of Contents

Submitted b……
Submitted to……
Abstract……..I
Acknowledgement……..II
Project Title……..1
Project Description……..1
Aim of the project……..2
Project Block Diagram……..3
Project Circuit Diagram……..4
Project Construction:……4
Hardware:……4
Simulation software:……..5
Project Operation -Program Description Language (PDL):……..5
Project Program Listing:……7
Description of the program:……12
Overall Structure:……12
Traffic Phase Timing:……12
Green Blink Warning:……..12
LCD Safety Messages:……..12
DHT11 Reading Strategy:……13
I2C LCD Protocol:……13
Reference:……14

Table of Figures

Figure1:BlockDiagram :……3
Figure2: ATmega32:……3
Figure3: LCD 16x2:……3
Figure 4: Circuit Diagram of the project:……4
Figure5: The Project Constructed on a breadboard:……4
Figure6: The Project Constructed on a Proteus:……5

ProjectTitle

Smart Traffic Light Control System with Environmental Monitoring usingATmega32

Project Description

This project presents the design and implementation of a Smart Traffic Light Control System that manages a two-signal road intersection using a single ATmega32 microcontroller operating at 8 MHz. The system is built to demonstrate the integration of real-time digital control, sensor interfacing, display multiplexing, and serial communication within a single embedded microcontroller platform.

The system controls two opposing traffic signals simultaneously. Each signal consists of three LEDs representing the standard Red, Yellow, and Green states. The signals are synchronized such that when Signal 1 is green, Signal 2 is red, and vice versa, following a realistic intersection timing pattern. A 40-second cycle governs each direction, with a 3-second yellow transition phase before every red phase.

A bank of four 7-segment display digits is used to show the countdown timer for both signals simultaneously. The digits are multiplexed on PORTA of the ATmega32, meaning all four digits share the same segment data bus and are switched on and off rapidly one at a time to create the illusion of a continuous fourdigit display.

An environmental monitoring subsystem is incorporated into the design using a DHT11 digital temperature and humidity sensor connected to pin PD2. The sensor readings are displayed in real time on a 16×2 character LCD module connected via the I2C protocol through a PCF8574 I/O expander. The upper row of the LCD shows the current temperature and humidity values, while the lower row cycles through four road safety advisory messages to alert drivers.

Aim of the Project

The primary aims of this project are:

1- To design and implement a functional two-signal traffic light controller on a single ATmega32 microcontroller without the need for any external timing ICs or multiple processors.

2- To demonstrate hardware multiplexing of a four-digit 7-segment display using only one 8-bit output port of the ATmega32.

3- To integrate a DHT11 digital sensor for real-time environmental monitoring and to display temperature and humidity data on an I2C LCD module.

4- To implement a software-based timing mechanism that produces accurate second intervals by combining a counting loop with the 7-segment multiplex duty cycle.

5- To implement a green-light blink warning feature during the final 3 seconds of each green phase to alert pedestrians and drivers of an upcoming phase change.

6- To cycle through public safety advisory messages on the LCD display, demonstrating the use of a traffic announcement subsystem within an embedded system.

7- To build a complete, well-structured embedded C program suitable for compilation in Atmel Studio 7 and execution on real ATmega32 hardware and Proteus simulation.

Project Block Diagram

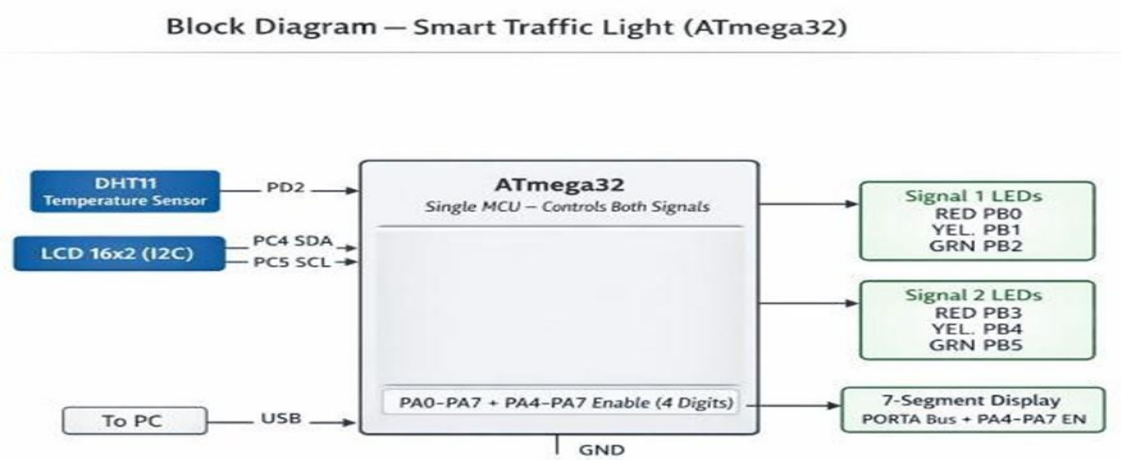
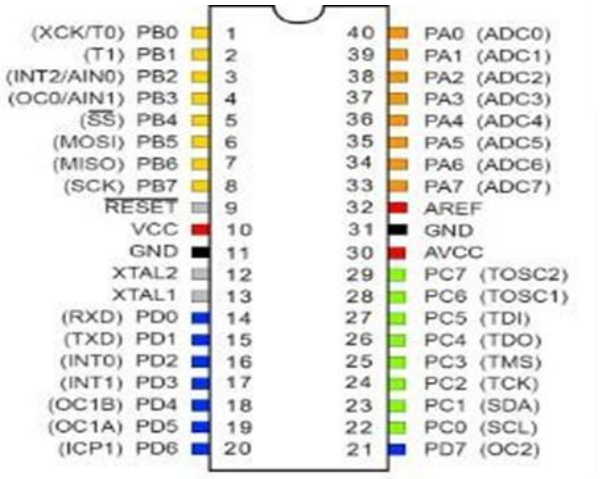


Figure1: Block Diagram

Show as Figure 1 System Block Diagram illustrating the integration of sensors, the ATmega32 microcontroller, and output peripherals for traffic and environmentalmonitoring.



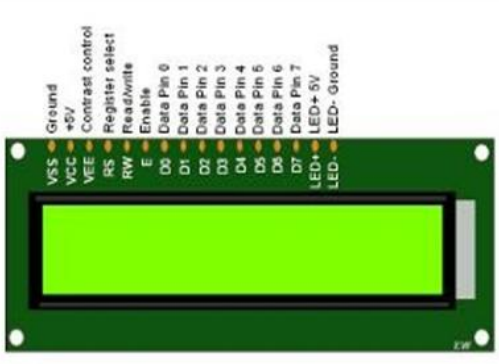


Figure2: ATmega32 *Figure3: LCD 16x2*

Show as Figure 2 ATmega32 Microcontroller The central processing unit utilized for real-time data execution and peripheral management.

Show as Figure 3 16x2 LCD Interface used to display real-time environmental data and the current status of the traffic control system

Project Circuit Diagram

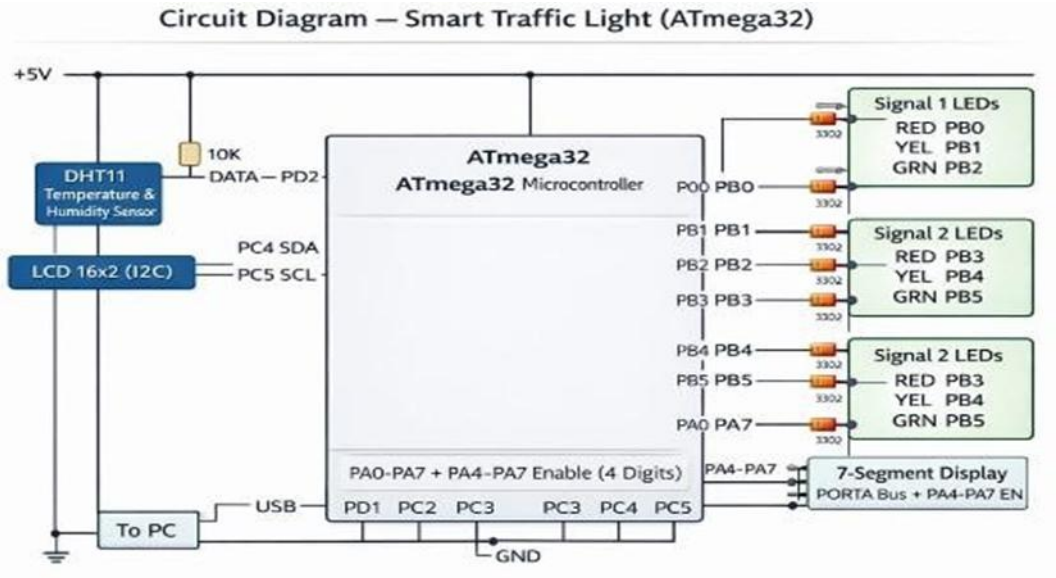


Figure 4:Circuit Diagram of the project

As illustrated in Figure 4 Detailed Schematic Diagram showing the electrical connections and interfacing between the ATmega32 and system components.

Project Construction

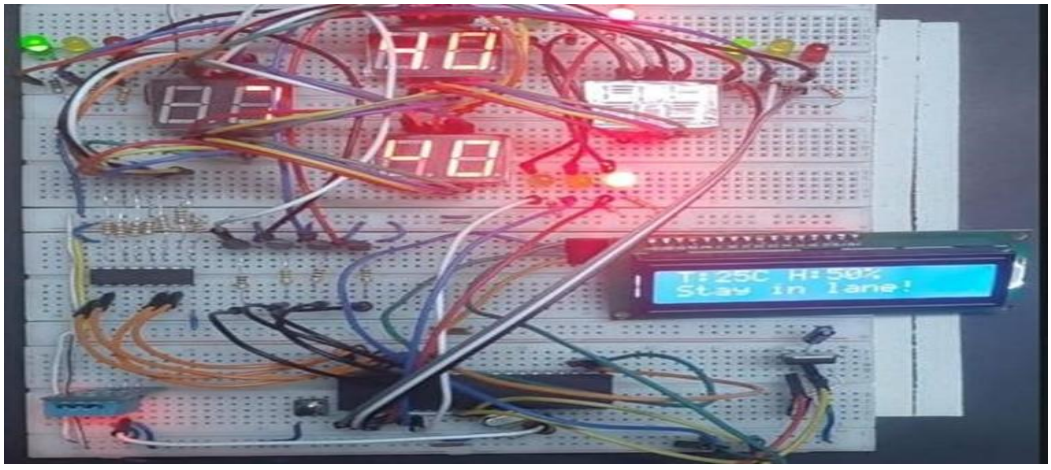


Figure5: The Project Constructed on a breadboard

Show as Figure 5: Physical Hardware Implementation showing the final prototype and the integration of environmental sensors with the traffic light LEDs.

Simulation software:

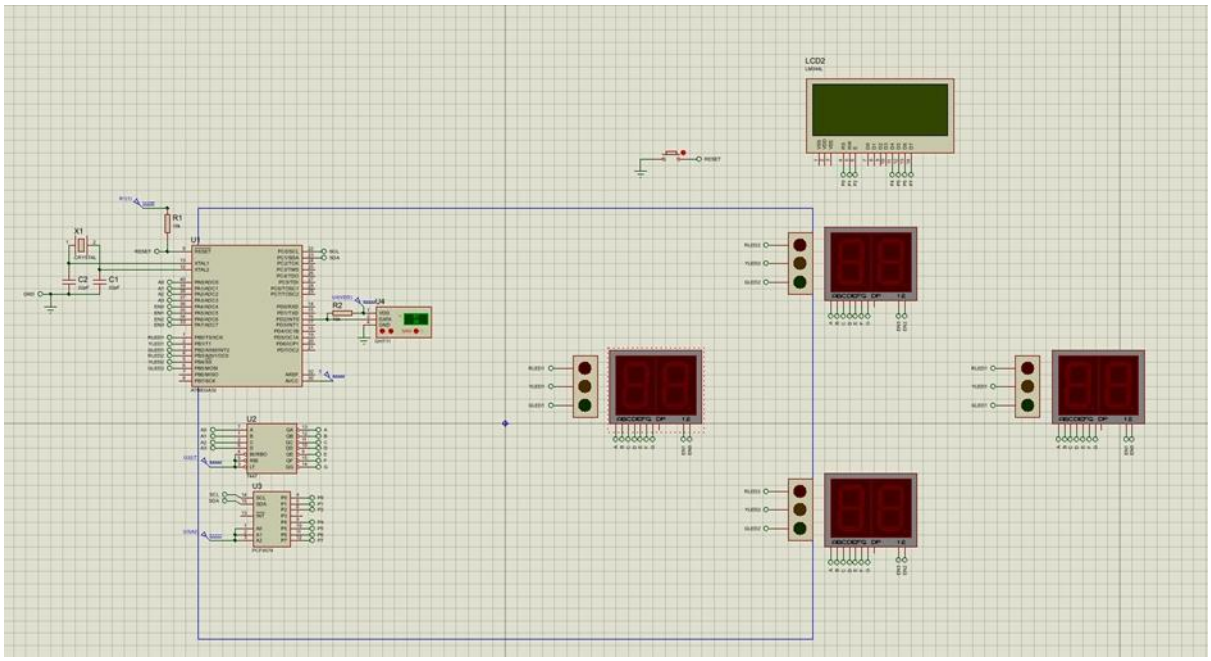


Figure 6: The Project Constructed on a Proteus

Show as Figure 6 Proteus Simulation environment demonstrating the functional validation of the control logic and hardware design.

Project Operation -Program Description Language (PDL)

The Program Description Language section describes the software logic of the entire system in structured English, mirroring the actual C source code. The PDL is organized to follow the execution flow from initialization through the main control loop and into each sub-procedure.

PDL:

START

Initialize ATmega32 ports

- PORTB → LEDs for Traffic Lights

- PORTA → 7-Segment display

- PORTD → DHT11 input

Initialize I2C

Initialize LCD 16x2

Display "System Starting..." on LCD

Initialize variables (Temperature, Humidity, Message Index)

LOOP FOREVER

// Step 1: Read Temperature & Humidity

Read DHT11 sensor

Store Temperature (I_Temp) and Humidity (I_RH)

Display "T: XXC H: XX%" on LCD (Line 1)

// Step 2: Display Warning Messages

Display message on LCD (Line 2) from list:

1. "Pedestrian Push Button"

2. "Night Mode (Low Traffic)"

3. "UART Monitoring"

4. "EEPROM Data Logging"

5. "Buzzer for Visually Impaired"

Cycle through messages every update

// Step 3: Traffic Light Sequence

// Road 1 Green / Road 2 Red

Turn ON GREEN1, RED2

Countdown on 7-Segment display

Blink LEDs if needed

Update LCD with messages

// Road 1 Yellow / Road 2 Red

Turn ON YELLOW1, RED2

Countdown on 7-Segment

Update LCD with messages

// Road 1 Red / Road 2 Green

Turn ON RED1, GREEN2

Countdown on 7-Segment

Blink LEDs if needed

Update LCD with messages

// Road 1 Red / Road 2 Yellow

Turn ON RED1, YELLOW2

Countdown on 7-Segment

Update LCD with messages

// Step 4: Repeat Loop

END LOOP

END

Project Program Listing

The following is the complete C source code of the project, written for Atmel Studio 7

with the AVR-GCC toolchain targeting the ATmega32 at 8 MHz. The code is

structured into initialization, I2C driver, LCD driver, DHT11 driver, multiplexing, and

the main control loop.

```
#define F_CPU 8000000UL

#include <avr/io.h>

#include <util/delay.h>

#include <stdio.h>
```

```
#define LCD_ADDR 0x4E
#define DHT_PIN PD2
uint8_t I_Temp = 25, I_RH = 50, msg_index = 0;
void i2c_init() {
TWSR = 0x00;
TWBR = 0x20;
TWCR = (1 << TWEN);
}
void i2c_start() {
TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
while (!(TWCR & (1 << TWINT)));
}
void i2c_stop() {
TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
}
void i2c_write(uint8_t data) {
TWDR = data;
TWCR = (1 << TWINT) | (1 << TWEN);
while (!(TWCR & (1 << TWINT)));
}
void lcd_send(uint8_t val, uint8_t mode) {
uint8_t high = (val & 0xF0) | mode | 0x08;
uint8_t low = ((val << 4) & 0xF0) | mode | 0x08;
i2c_start();
i2c_write(LCD_ADDR);
i2c_write(high | 0x04); _delay_us(50); i2c_write(high & ~0x04);
```

```
i2c_write(low | 0x04); _delay_us(50); i2c_write(low & ~0x04);
i2c_stop();
}
void lcd_init_i2c() {
_delay_ms(50);
lcd_send(0x03, 0); _delay_ms(5);
lcd_send(0x03, 0); _delay_us(150);
lcd_send(0x03, 0);
lcd_send(0x02, 0);
lcd_send(0x28, 0);
lcd_send(0x0C, 0);
lcd_send(0x01, 0);
_delay_ms(2);
}
void lcd_string(const char *s) {
while (*s) lcd_send(*s++, 1);
}
void read_dht11() {
uint8_t bits[5] = {0,0,0,0,0}, i, j, timeout;
DDRD |= (1 << DHT_PIN); PORTD &= ~(1 << DHT_PIN); _delay_ms(18);
PORTD |= (1 << DHT_PIN); DDRD &= ~(1 << DHT_PIN); _delay_us(30);
timeout = 200; while ((PIND & (1 << DHT_PIN)) && --timeout);
if (timeout == 0) return;
timeout = 200; while (!(PIND & (1 << DHT_PIN)) && --timeout);
timeout = 200; while ((PIND & (1 << DHT_PIN)) && --timeout);
for (i = 0; i < 5; i++) {
```

```
for (j = 0; j < 8; j++) {
timeout = 200; while (!(PIND & (1 << DHT_PIN)) && --timeout);
_delay_us(35);
if (PIND & (1 << DHT_PIN)) bits[i] |= (1 << (7 - j));
timeout = 200; while ((PIND & (1 << DHT_PIN)) && --timeout);
}
}
if (bits[0] != 0 || bits[2] != 0) { I_RH = bits[0]; I_Temp = bits[2]; }
}
void update_info() {
char buf[20];
const char* msgs[] = {"Stay in lane ", "Fasten seatbelt ", "No phone use ",
"Focus on road "};
read_dht11();
lcd_send(0x80, 0);
sprintf(buf, "T:%02dC H:%02d%% ", I_Temp, I_RH);
lcd_string(buf);
lcd_send(0xC0, 0);
lcd_string(msgs[msg_index]);
msg_index = (msg_index + 1) % 4;
}
void multiplex_pnp(uint8_t n1, uint8_t n2) {
uint8_t digits[] = {n1 / 10, n1 % 10, n2 / 10, n2 % 10};
uint8_t select_mask[] = {0xE0, 0xD0, 0xB0, 0x70};
for (int i = 0; i < 4; i++) {
PORTA = 0xF0;
_delay_us(100);
```

```
PORTA = (digits[i] & 0x0F) | select_mask[i];
_delay_ms(3);
}
}
void count_one_sec(uint8_t t1, uint8_t t2, int blink_pin) {
update_info();
for (int j = 0; j < 65; j++) {
if (blink_pin != -1 && j % 20 == 0) PORTB ^= (1 << blink_pin);
multiplex_pnp(t1, t2);
}
}
int main(void) {
DDRA = 0xFF;
DDRB = 0xFF;
DDRC = 0xFF;
DDRD &= ~(1 << DHT_PIN);
PORTA = 0xF0;
i2c_init();
_delay_ms(100);
lcd_init_i2c();
while (1) {
PORTB = (1 << PB2) | (1 << PB3);
for (int i = 37; i > 4; i--) count_one_sec(i, i + 3, -1);
for (int i = 4; i > 0; i--) count_one_sec(i, i + 3, PB2);
PORTB = (1 << PB1) | (1 << PB3);
for (int i = 3; i > 0; i--) count_one_sec(i, i, -1);
```

```
PORTB = (1 << PB0) | (1 << PB5);

for (int i = 37; i > 4; i--) count_one_sec(i + 3, i, -1);

for (int i = 4; i > 0; i--) count_one_sec(i + 3, i, PB5);

PORTB = (1 << PB0) | (1 << PB4);

for (int i = 3; i > 0; i--) count_one_sec(i, i, -1);

}

}
```

Description of the Program

Overall Structure:

The program follows a simple super-loop (bare-metal) architecture with no operating system or interrupt-driven scheduling. All timing is achieved through software delay loops combined with the 7-segment multiplex cycle. The main function initializes all peripherals then enters an infinite while loop that sequences through the four traffic phases repeatedly.

Traffic Phase Timing:

Each complete intersection cycle lasts approximately 86 seconds: 40 seconds for Phase 1 (Signal 1 green), 3 seconds for Phase 2 (Signal 1 yellow), 40 seconds for Phase 3 (Signal 2 green), and 3 seconds for Phase 4 (Signal 2 yellow). The countdown displayed on the 7-segment digits reflects the remaining time in each individual phase. Signal 1 shows its own phase count on the left two digits, and Signal 2 shows its phase count on the right two digits.

Green Blink Warning:

During the final 3 seconds of any green phase, the blinkPin parameter of

countDownOneSec is set to the corresponding green LED pin. Inside the multiplex loop, the LED is toggled every 30 iterations (approximately every 270 ms), producing a visible on-off blink that warns drivers the signal is about to change to yellow. This is implemented using the XOR toggle on PORTB:

PORTB ^= (1 << blinkPin).

LCD Safety Messages: Four safety advisory messages are stored as constant strings in program memory.

The msg_index variable selects the current message and is incremented modulo 4 on every call to update_system, which is called once per second. This means the message changes every second, cycling through: 'Staying in lane', 'Fasten seatbelt', 'No phone driving', and 'Focus on the road'.

DHT11 Reading Strategy:

The DHT11 is read once per second inside update_system. If the sensor fails to

respond (line stays high after the start condition) or returns all-zero data, the global variables I_Temp and I_RH retain their last valid values, so the LCD always shows a meaningful reading. This prevents display glitches due to occasional missed sensor reads.

I2C LCD Protocol:

The lcd_send function sends one byte to the LCD in 4-bit mode through the

PCF8574 I2C expander. Each byte requires one I2C transaction containing four write operations: high nibble with EN high, high nibble with EN low, low nibble with EN high, and low nibble with EN low. The backlight bit (BL_BIT = bit 3 of the PCF8574 output) is set in every write to keep the backlight on continuously.

Reference

1- Microchip Technology Inc., "ATmega32 / ATmega32A 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash — Datasheet," Microchip Technology, Doc. No.DS40002060A, 2020. Available:

https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega32A-DataSheet-Complete-DS40002072A.pdf

2- D. W. Smith, "AVR Microcontroller and Embedded Systems," in "Embedded Systems

Design," J. K. Valvano, Ed. Austin, TX: University of Texas Press, 2012, ch. 4, pp. 102–158.

3- Microchip Developer Help, "Using the TWI Module of the AVR as I2C Master," Microchip,

Tech. Note TB3215, 2020. Available: https://developer.microchip.com